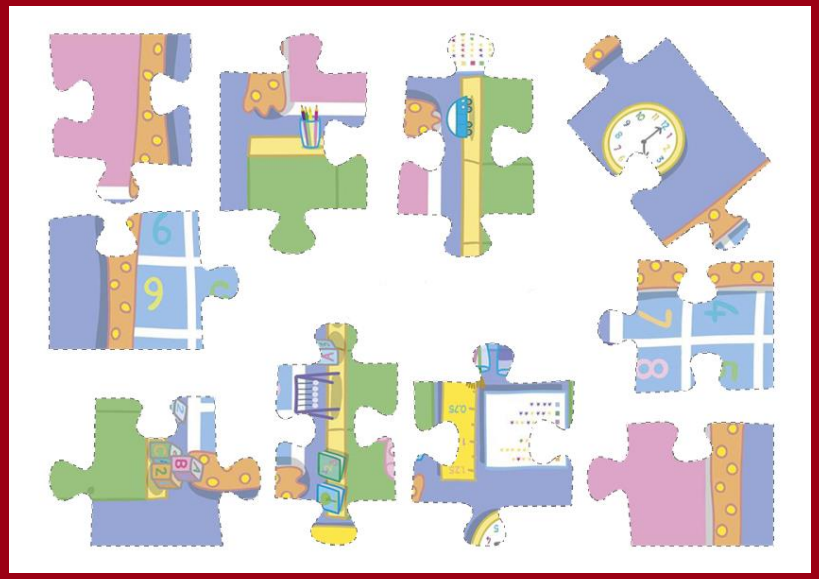
DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

UNIVERSITÀ
DEGLI STUDI
DI PADOVA



LAB 3: Solve the Puzzle!

Computer Vision 2024-25

P. Zanuttigh, F. Barbato, M. Caligiuri

CV 2024: HW / Projects

1. Introduction to C++ / OpenCV

No HW

2. **Histogram eq. and Filtering**
3. Road Line Detection

- Select 1 among LAB1 and LAB2
- First «simple» HW: 2 points
- Provide a very short report or place comments in the source
- On/off mark
- Deadline 29/11

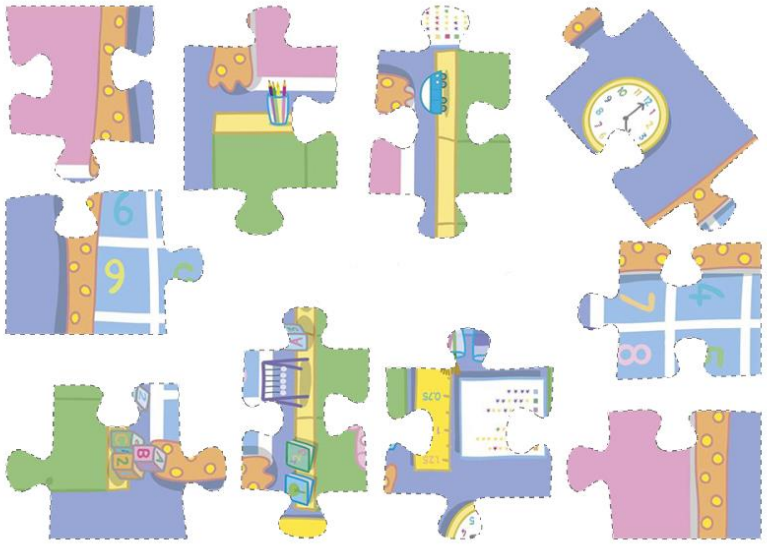
4. **Features Matching**
5. **Deep Learning**

- Select 1 among LAB3 and LAB4
- Second «advanced» HW: 5 points
- More detailed report with results
- Mark based on solution quality

Final Exam (written in classroom): ~25 points

Final mark: **2** + 5 + 25 = max 32 points

CV 2024: Final Projects

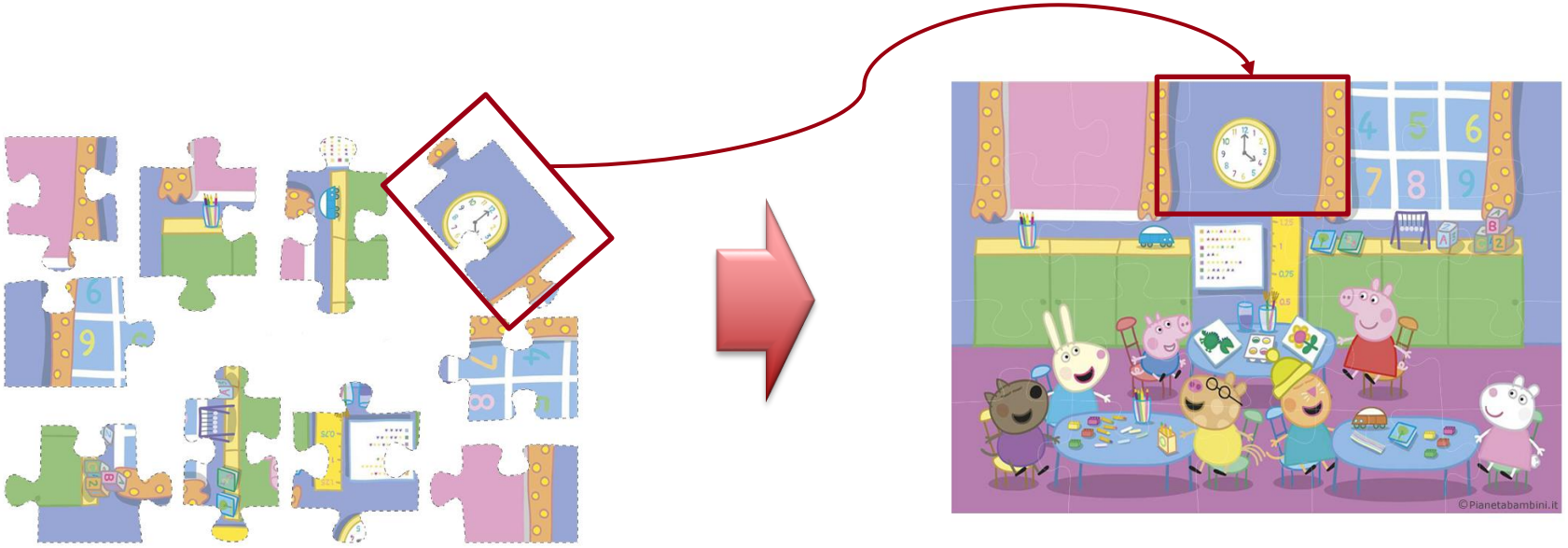




Two options:

1. ***Solve the Puzzle (LAB3)***
2. Detect traffic light and road signs (*LAB4*)

PROJECT 1: SOLVE THE PUZZLE



- Try to re-assemble the pieces to complete the puzzle
- Segment the various pieces from the provided images
- Use the full puzzle image as reference to get the position of each piece (exploiting feature matching or any other technique!)
- Combine the pieces to re-create the puzzle

Algorithm to be Developed

1. Load the patches' images and the reference image with the full puzzle
   - You can use one of the provided sets: the "*peppa*" one is simpler (except one patch that is a bit more difficult), while the "*city_blue*" is slightly more challenging, the "*city_black*" is the most challenging, no need to completely solve all 3.
   - Each set contains a reference image and collections of patches (you can use subset of patches in the "*piecesX*" images or the full set in the "*pz*" ones
   - Notice that the patches can be translated/rotated/scaled
2. Segment the patches from the provided image (you can use any segmentation approach and/or thresholding, connected components or object detection algorithms)
3. Extract features from each patch (*e.g., SIFT or ORB*)
4. Extract features from the reference image (*e.g., SIFT or ORB*)
5. Locate each patch on the reference image
   - Compute the matches between the patch and image features
   - You can use the cv::BFMatcher class (use default L2 distance for SIFT)
   - You can refine the found matches by selecting the matches with distance less than *ratio*min_distance*, where *ratio* is a user-defined threshold and *min_distance* is the minimum distance found among the matches
6. You can assume the images and patches are linked by an affine transform
   - Using the refined matches find the transformation between the images
   - You can use the RANSAC algorithm implemented into OpenCV *findHomography()*
7. Create a new image with all the patches over it

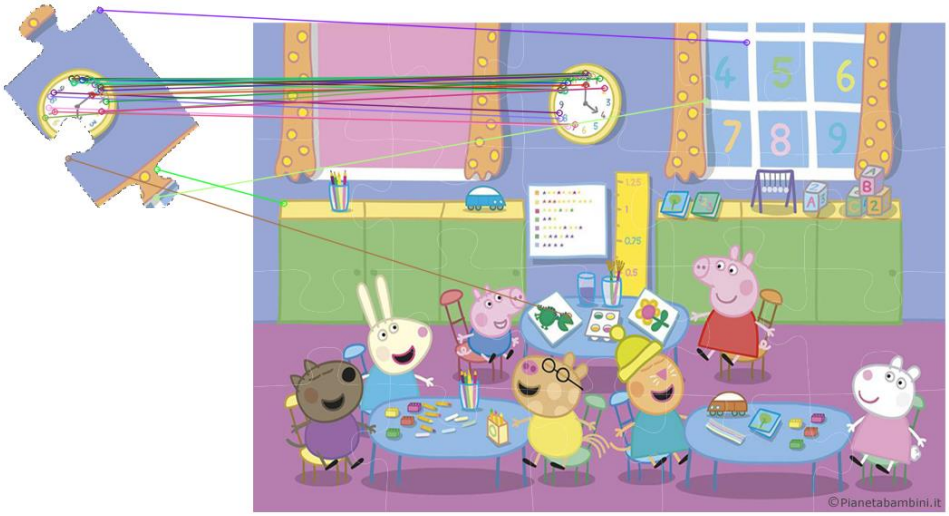
This is the baseline assignment, see the next slide for additional suggestions for extra features improving your mark

Extra Ideas (optional)

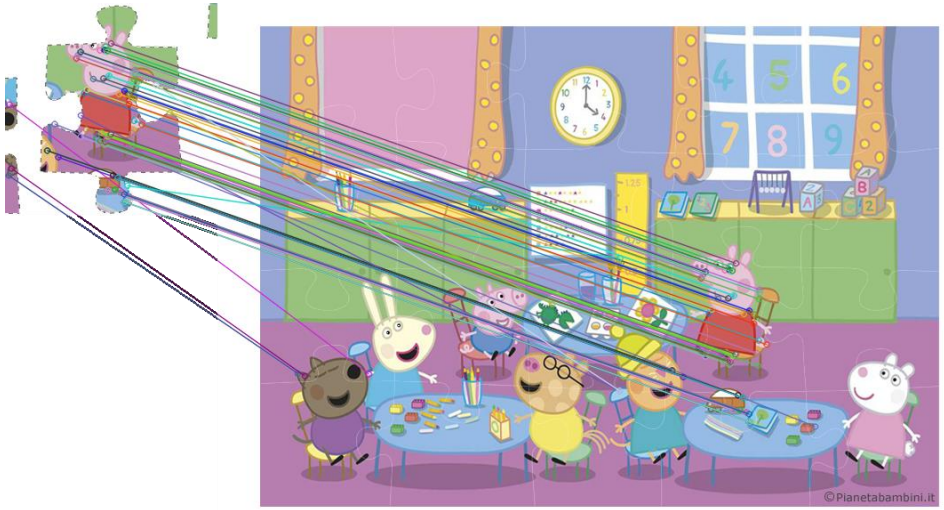
1. Manually implement RANSAC and affine transform estimation
2. Try various algorithms/strategies to segment the patches, e.g., segmentation, thresholding, morphological operators, object detectors…
3. Try different feature descriptors (e.g., ORB) or try to match in other ways, e.g. using contours
4. Use some blending/mixing technique to improve final image quality
5. Use other techniques for the matching, e.g., template matching or machine learning based approaches
6. Test other images
7. *(very challenging!)* Try to solve without using the reference image
8. Any additional idea is welcome !

Differently from HW1, this final project lab has a mark,
a few of these or any other «extra» idea could improve your mark

Example of Matching and Localization









OpenCV: Feature2D Class

virtual void cv::Feature2D:: detectAndCompute	(	InputArray	image,	Input image
		InputArray	mask,	Compute KP only in regions where mask is not 0
		std::vector< KeyPoint > &	keypoints,	Output keypoints (location, orientation, scale)
		OutputArray	descriptors,	KP descriptors
		bool	useProvidedKeypoints = false	Set to false
	)			

Base class for feature extractor and descriptors

Three methods:

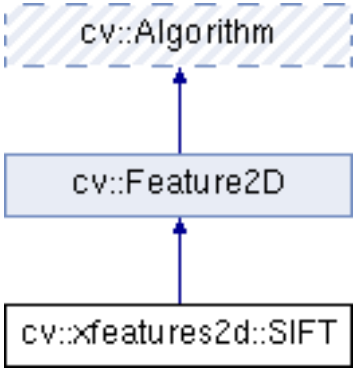
- *detect* (feature extraction)
- *compute* (feature description)
- *detectAndCompute* (both stages)

Constructor depends on the employed subclass

➢ e.g., SIFT, ORB, etc… , each method has its own specific parameters

SIFT in OpenCV

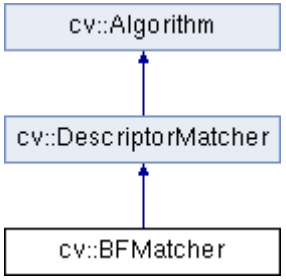
```
static Ptr<SIFT>
cv::SIFT::create   (  int      nfeatures = 0,                  # of feature points to extract
                      int      nOctaveLayers = 3,              #  of layers in each octave
                      double   contrastThreshold = 0.04,       Threshold on D(x̂)
                      double   edgeThreshold = 10,             Threshold on eigenvalue ratio
                      double   sigma = 1.6                     Smoothing of the 1st image 1st octave
                   )
```



Usually better performances than ORB

FEATURE MATCHING

```
static Ptr<BFMatcher>
cv::BFMatcher::create ( int      normType = NORM_L2,
                        bool     crossCheck = false
                      )
```



- Brute-Force matching
- Select the type of distance function
  - Use *NORM_L2* for SIFT
  - Use *NORM_HAMMING* for ORB
- *crossCheck* : if *true* forces that if A matches B, then B must match A
- Suggestion: refine keep only matches with a distance smaller than a threshold

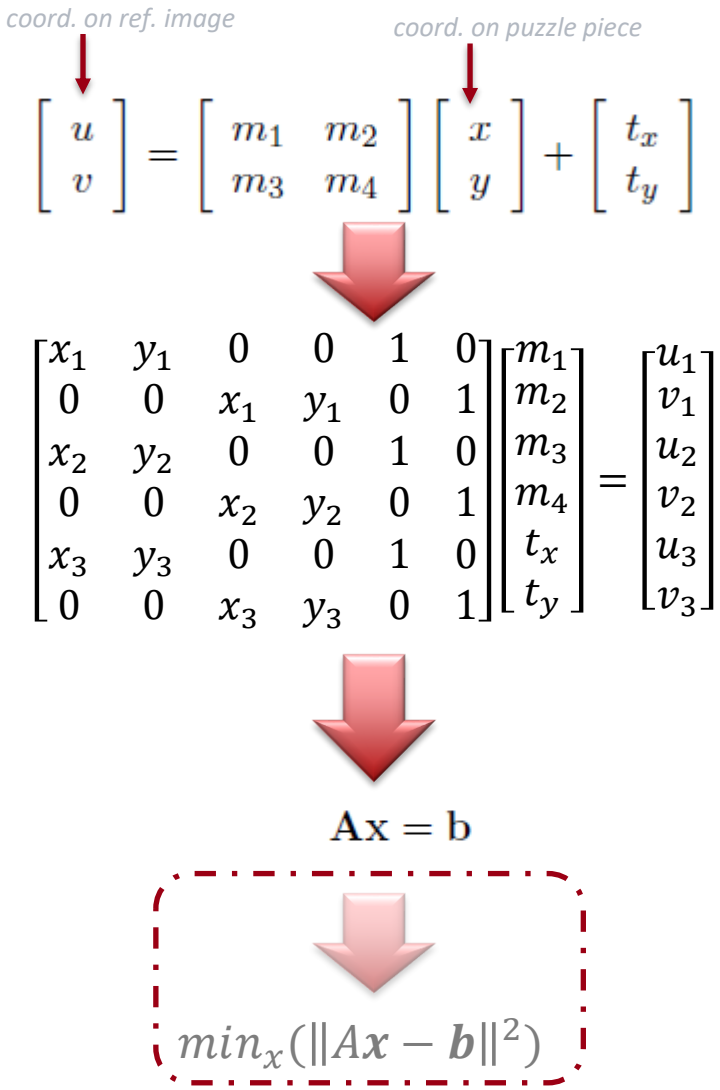
cv::findHomography

Mat cv::findHomography (	InputArray	srcPoints,	Coordinates of the points in the 1st image, a matrix of type CV_32FC2 or a vector<Point2f>
	InputArray	dstPoints,	Coordinates in the 2° image
	OutputArray	mask = noArray(),	Not needed
	int	method = 0,	Use cv::RANSAC
	double	ransacReprojThreshold = 3,	Threshold on reprojection error for a point to be considered an inlier
)			

- Finds the homography between object and its location in the image
- Returns a 3x3 matrix with the computed homography

$$s \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} \sim H \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

ESTIMATE THE AFFINE TRANSFORM



- Use the OpenCV *findHomography()* function
- It estimates a more general transformation than the affine one
- 8 parameters (DoF) instead of 6
- Includes the RANSAC algorithm

- Otherwise (*advanced, optional*) implement manually a simplified RANSAC estimator
  - Try different random sets of 3 points
  - For each set of 3 KP estimate the corresponding affine transform
  - Find the one with the largest consensus set
- Simpler (but slower) than the Hough Transform approach seen in the theory, but enough for simple cases

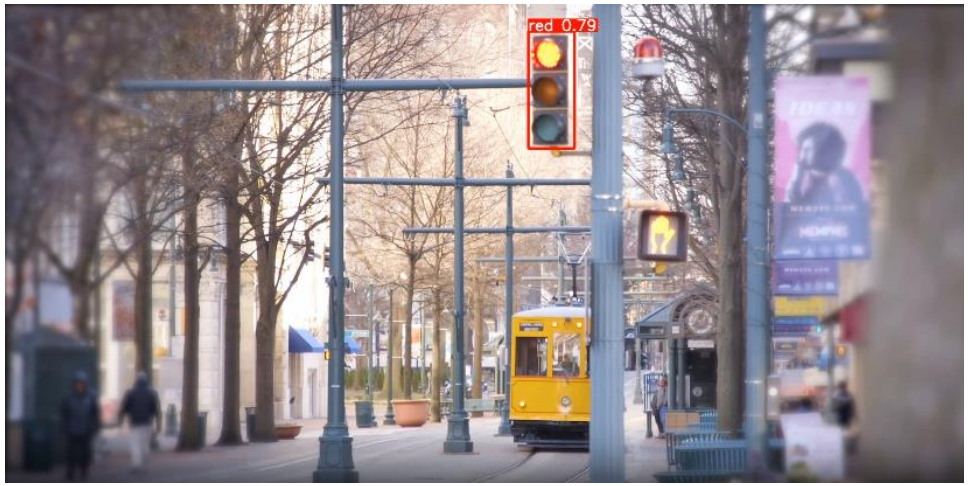
RANSAC (*SIMPLIFIED*)

1. Select a random set of 3 correspondences
2. Estimate the affine transform corresponding to them as in previous slide
3. Count how many correspondences are consistent with the selected one. The criteria can be a threshold on the difference between the projections of the keypoint and their locations, i.e., $|\Delta u_{max}|+|\Delta v_{max}|<T$ *(e.g., T=3)*
4. Iterate *n* times (e.g., *n*=100) and keep the correspondence with the largest compatible set

Included inside openCV findHomography()

Project 2: Road Signs and Traffic Lights Detection





- Detect road signs and traffic lights
- For each sign also classify it into the proper category (e.g., speed limit, stop, etc)
- Detect if the traffic light is green, yellow or red
- For speed limits read the maximum speed value
- *Support in LAB4 but data will be provided soon*
- *More details in one of the next lectures*

Algorithm to be Developed (Draft)

1. Load some of the provided images
2. Find the road signs and the traffic lights (you can use any object detection strategy, e.g., Hough Transform for specific shapes, bag-of-words, deep learning-based object detectors)
3. For each traffic light analyze it to get if it is green, yellow or red
4. For speed limit signs find the central region and analyze it to read the maximum allowed speed value

More details in next lectures and in LAB4

Sample Data for Project2







A zip file is provided on elearning with a few sample images (more data will be provided later

Have a look at this two datasets

- (traffic lights) https://zenodo.org/records/12706046
- (road signs) https://www.cvl.isy.liu.se/research/datasets/traffic-signs-dataset/